

Module 4

Memory Management

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

Memory Management

- Memory management is a crucial function of an operating system (OS) that handles the primary memory (RAM).
- It's responsible for efficiently managing the memory, allocating it to processes, and deallocating it when processes terminate.
- This ensures that multiple processes can coexist and run smoothly without interfering with each other's memory space.

Memory Management

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of **addresses + read** requests, or **address + data and write** requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management.
 - **Logical address** – generated by the CPU; also referred to as **virtual address**.
 - **Physical address** – address seen by the memory unit.
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme.
- **Logical address space** is the set of all logical addresses generated by a program.
- **Physical address space** is the set of all physical addresses generated by a program.

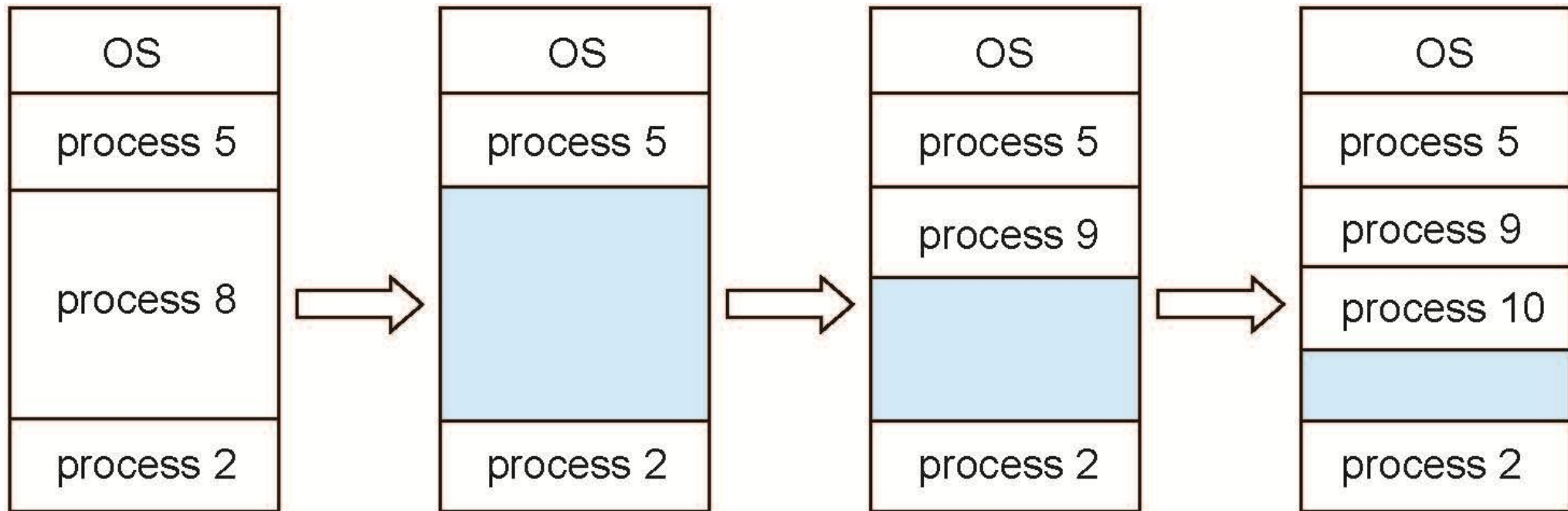
Contiguous Allocation

- Main memory must support both OS and user processes.
- Limited resource, must allocate efficiently.
- Contiguous allocation is one early method.
- Main memory usually into two **partitions**:
 - Resident operating system, usually held in low memory with interrupt vector.
 - User processes then held in high memory.
 - Each process contained in single contiguous section of memory.

Multiple-partition allocation

- Multiple-partition allocation
 - Degree of multiprogramming limited by number of partitions
 - **Variable-partition** sizes for efficiency (sized to a given process' needs)
 - **Hole** – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Process exiting frees its partition, adjacent free partitions combined
 - Operating system maintains information about:
 - a) allocated partitions b) free partitions (hole)

Multiple-partition allocation



Memory Allocation Algorithms

- When a process needs memory, the OS must find a suitable block of free space.
- Several algorithms are used for this purpose.
- Let's assume we have a list of free memory partitions (holes) and a process requesting a specific size of memory.

Dynamic Storage-Allocation Problem

- How to satisfy a request of size n from a list of free holes?
 - **First-fit**: Allocate the *first* hole that is big enough.
 - **Best-fit**: Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size.
 - Produces the smallest leftover hole.
 - **Worst-fit**: Allocate the *largest* hole; must also search entire list.
 - Produces the largest leftover hole.
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization.

Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation.
 - $1/3$ may be unusable -> **50-percent rule**.

First Fit

- **Concept:** The simplest algorithm. It scans the memory from the beginning and allocates the first hole that is big enough to fit the process.
- **Pros:** Fast and easy to implement.
- **Cons:** Tends to leave small, unusable fragments at the beginning of the memory, which can lead to inefficient use of space.

Best Fit

- **Concept:** This algorithm searches the entire list of free holes and allocates the smallest hole that is big enough for the process. This seems counter-intuitive, but the goal is to leave the largest possible free holes for future, bigger processes.
- **Pros:** Generally, leads to the most efficient use of memory by avoiding fragmentation of large holes.
- **Cons:** Slower than First Fit because it has to search the entire list of holes. It can still create many small, unusable fragments (external fragmentation).

Worst Fit

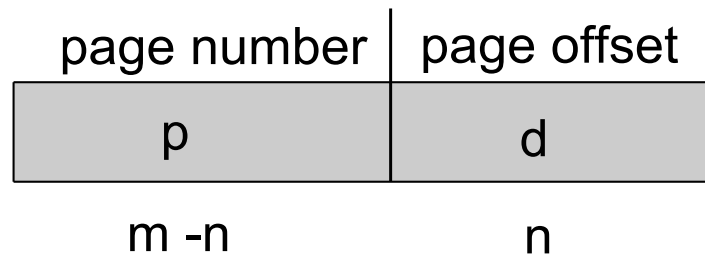
- **Concept:** The opposite of Best Fit. It allocates the process to the largest available hole.
- **Pros:** Aims to leave a large remaining hole after allocation, which might be useful for another large process.
- **Cons:** Can lead to poor memory utilization, as it breaks up the largest free spaces, potentially preventing any large future processes from being allocated.

Paging

- Paging is a memory management scheme that allows to allocate a process's physical memory in a **non-contiguous** way.
- It divides the logical address space of a process into equal-sized blocks called **pages**.
- The physical memory is also divided into equal-sized blocks of the same size, called **frames**.
- The OS maintains a page table for each process. This table maps each page of a process to a frame in physical memory. When a process requests a memory address, the CPU uses the page table to translate the logical address into a physical address.

Paging

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a page table which contains base address of each page in physical memory
 - **Page offset (d)** – (The location within the page) combined with base address to define the physical memory address that is sent to the memory unit



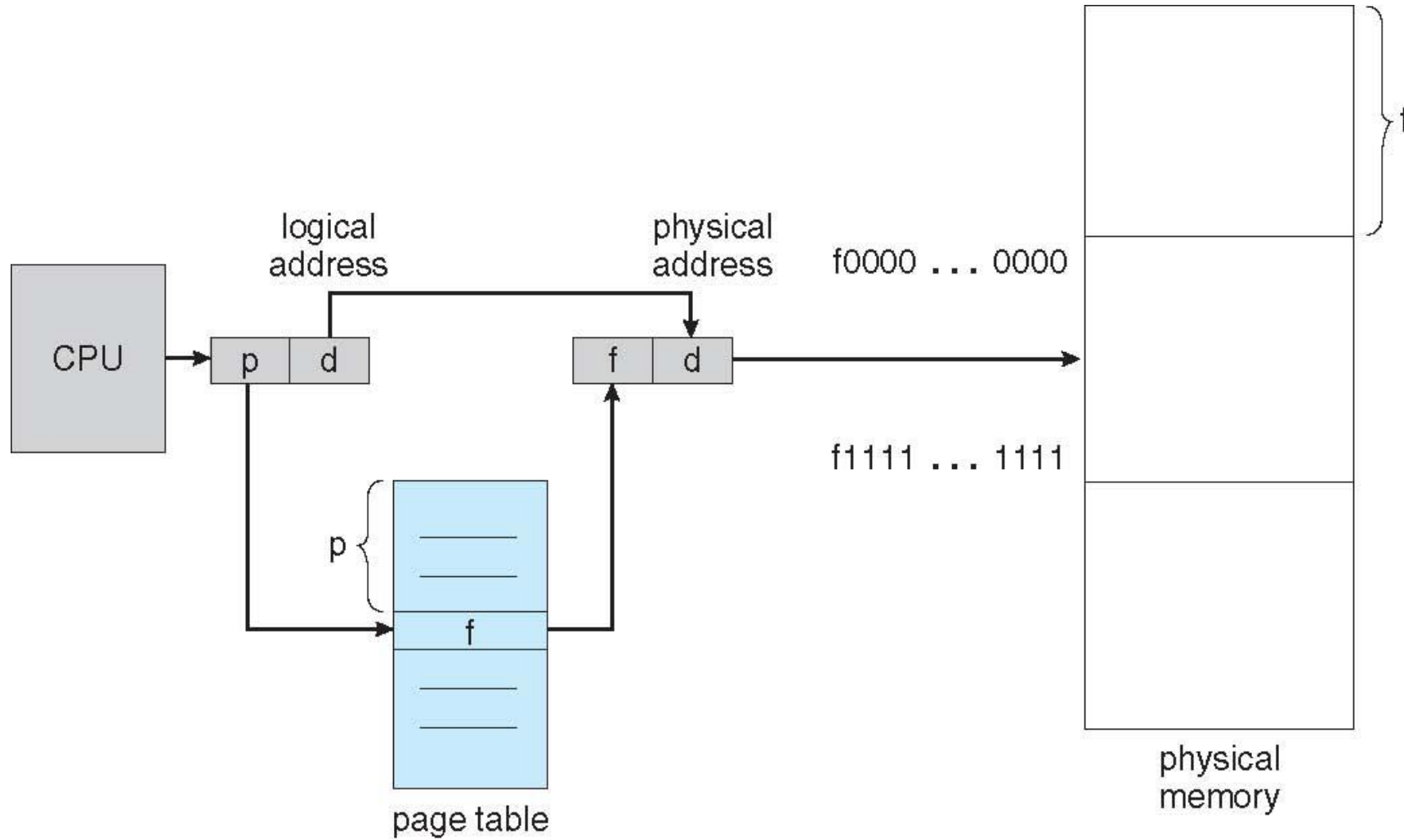
- For given logical address space 2^m and page size 2^n

$$\text{Physical Address} = (\text{Frame Number} * \text{Frame Size}) + \text{Page Offset}$$

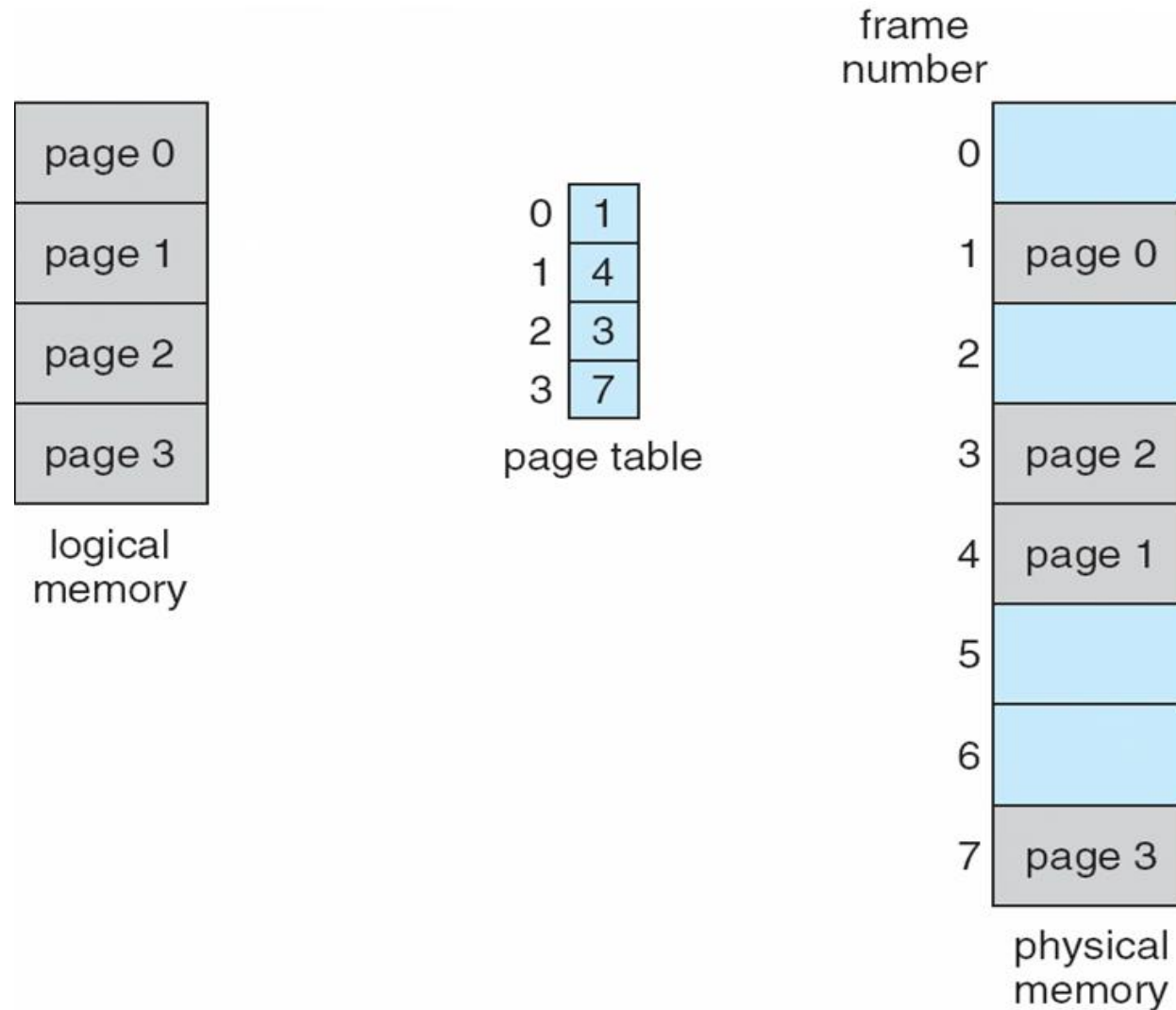
Paging

- Logical to Physical Address Translation (Paging):
 - A logical address is generated by the CPU.
 - The logical address is split into a page number and an offset.
 - The page number is used as an index into the process's page table.
 - The page table entry for that page number contains the corresponding frame number.
 - The frame number is combined with the offset to form the final physical address, which is then sent to the memory unit.

Paging Hardware



Paging Model of Logical and Physical Memory



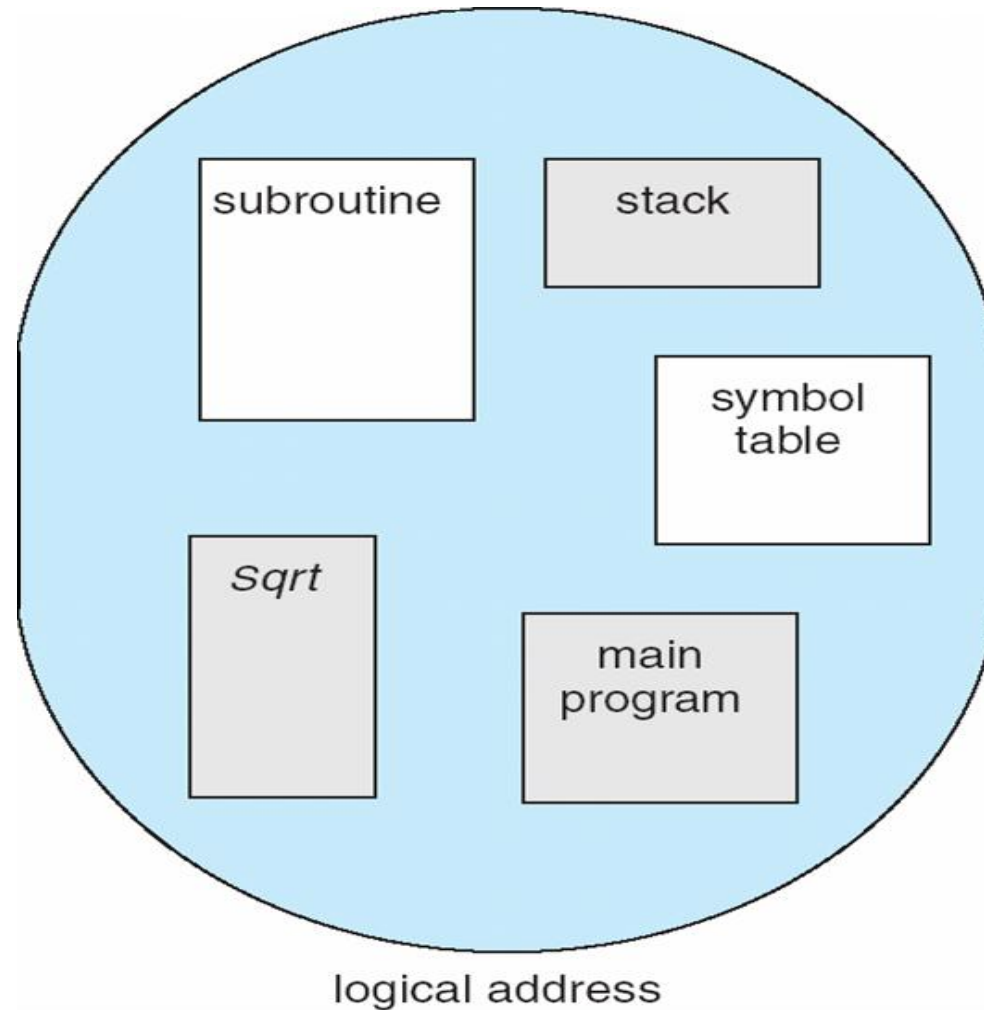
Paging

- Calculating internal fragmentation
 - Page size = 2,048 bytes
 - Process size = 72,766 bytes
 - 35 pages + 1,086 bytes
 - Internal fragmentation of $2,048 - 1,086 = 962$ bytes
 - Worst case fragmentation = 1 frame – 1 byte
 - On average fragmentation = $1 / 2$ frame size
 - So small frame sizes desirable?
 - But each page table entry takes memory to track
 - Page sizes growing over time
 - Solaris supports two page sizes – 8 KB and 4 MB
- Process view and physical memory now very different
- By implementation process can only access its own memory

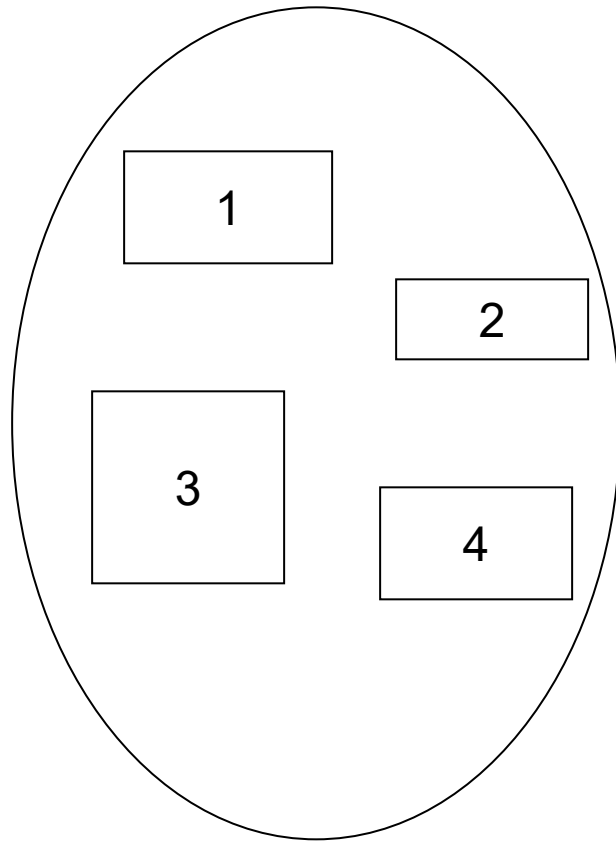
Segmentation

- Segmentation is a memory management scheme that allows the user's view of memory to be closer to their logical view of the program.
- The program is divided into variable-sized logical units called segments (e.g., code, data, stack, subroutines). Each segment is a contiguous block of memory.
- A segment table is used for address translation. Each entry in the segment table has a segment base address (the starting physical address) and a segment limit (the length of the segment).

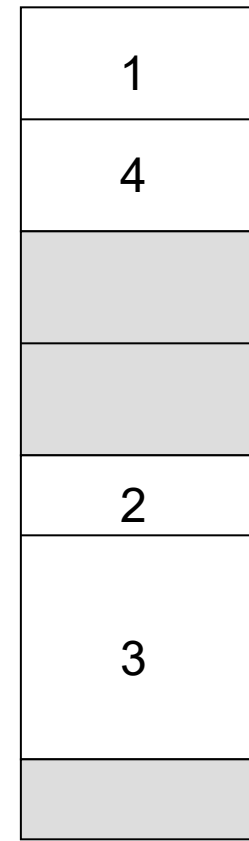
User's View of a Program



Logical View of Segmentation



user space



physical memory space

Segmentation

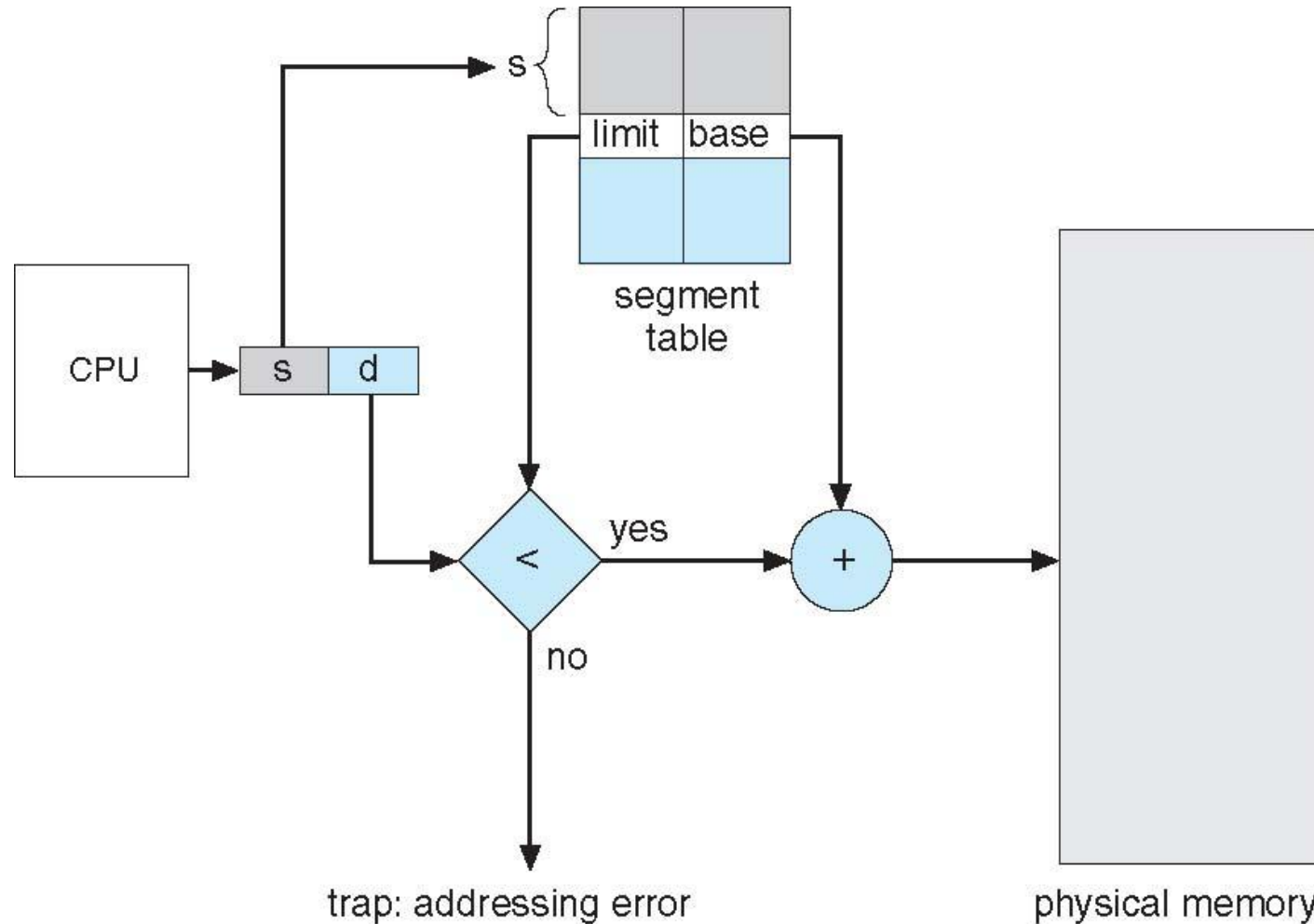
- **Logical Address:** Composed of two parts:
 - **Segment number (s):** An index into the segment table.
 - **Segment offset (d):** The location within the segment.
- **Physical Address:** The offset is added to the base address of the segment.

$$\text{Physical Address} = \text{Segment Base} + \text{Segment Offset}$$

Segmentation

- Logical to Physical Address Translation (Segmentation):
 - A logical address is generated by the CPU.
 - The logical address is split into a segment number and an offset.
 - The segment number is used as an index into the process's segment table.
 - The CPU checks if the offset is less than the segment limit (a protection mechanism).
 - If valid, the base address from the segment table is added to the offset to get the physical address.

Segmentation Hardware



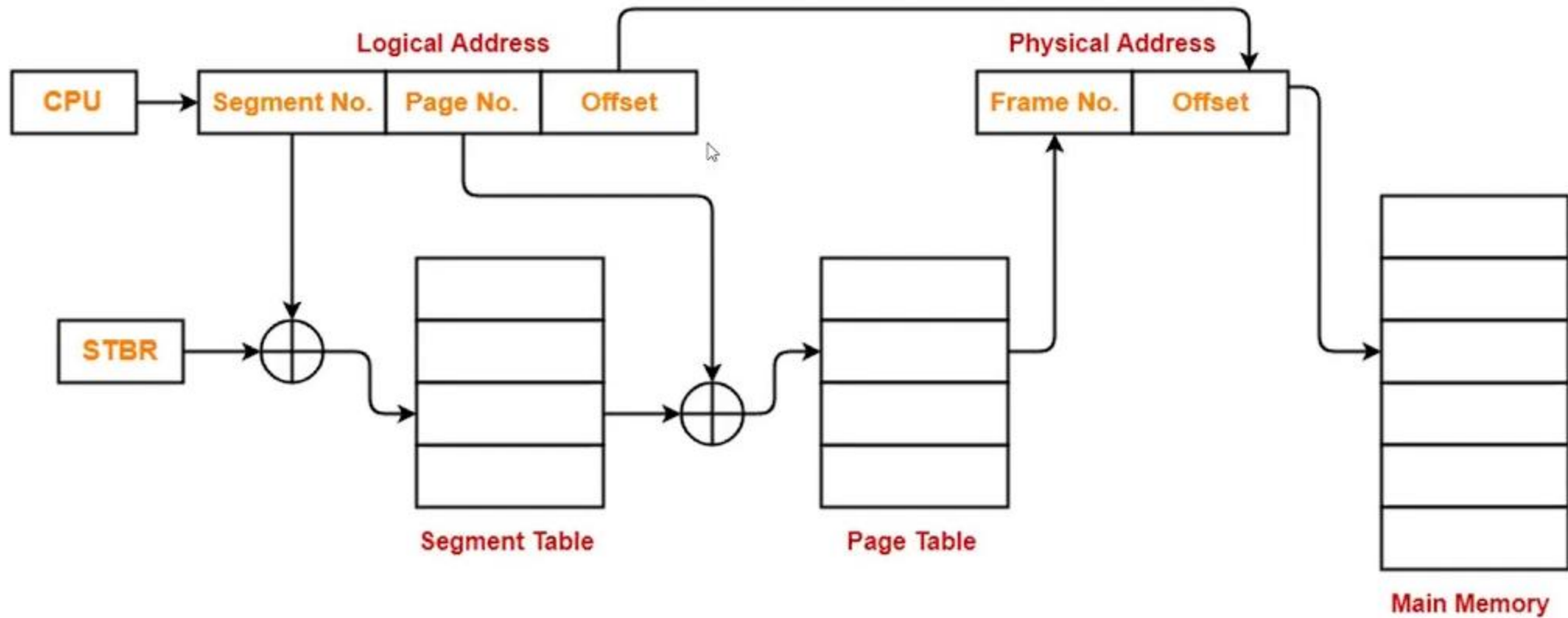
Paging with Segmentation (Segmented Paging)

- This is a hybrid approach that combines the benefits of both schemes.
- The logical address space is divided into segments, and each segment is then divided into pages. This is a common implementation in modern operating systems.
- A two-level address translation is required.
 - The CPU generates a logical address with a segment number, a page number, and an offset.
 - The segment number is used to look up a segment table. Each entry in the segment table points to a page table for that specific segment.
 - The page number is then used as an index into the segment's page table to find the corresponding frame number.
 - Finally, the frame number is combined with the offset to get the physical address.

Paging with Segmentation (Segmented Paging)

- Logical to Physical Address Translation (Segmented Paging):
 - A logical address is generated by the CPU, consisting of a segment number, a page number, and an offset.
 - The segment number is used to index into the segment table.
 - The segment table entry provides the base address of the segment's page table.
 - The page number is used to index into the retrieved page table. The page table entry provides the frame number.
 - The frame number is combined with the offset to form the final physical address.

Paging with Segmentation



Translating Logical Address into Physical Address

Paging with Segmentation

Feature	Description	Benefit
Paging Benefits	Eliminates External Fragmentation by dividing physical memory into small, fixed-size frames (pages).	Efficient use of physical memory (RAM).
Segmentation Benefits	Provides a clear logical view of memory (e.g., Code, Data, Stack) and is excellent for protection and sharing between processes.	Simplifies programming and enhances system security.
Combined	Segments can grow and shrink dynamically without issues because they are composed of pages that can be placed non-contiguously in memory.	Maximum flexibility for memory allocation and process resizing.

End of Module 4